



AMITY UNIVERSITY
— UTTAR PRADESH —

Internship Report on

**Centralized vs Peer-to-Peer Open-Source VPN Architectures
in Linux Environments**

Submitted to

Amity School of Engineering & Technology, ASET

In partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology (Information Technology)

Submitted To
Dr. Veenu Singh

Submitted By
Samay Kumar
Enroll No: A2305325001
Batch: 2025-2029

DECLARATION

I, Samay Kumar, student of Amity School of Engineering & Technology, hereby declare that the Internship-I report titled “Centralized vs Peer-to-Peer Open-Source VPN Architectures in Linux Environments”, submitted by me to Dr. Veenu Singh the Department of Information Technology, ASET, Amity University Uttar Pradesh, Noida, in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology (Information Technology), has not previously formed the basis for the award of any degree, diploma, or other similar title or recognition.

The Author attests that permission has been obtained for the use of any copyrighted material appearing in this report other than brief excerpts requiring only proper acknowledgement in scholarly writing and that all such use has been duly acknowledged.

Place: Noida

Date:

(Signature)

Samay Kumar

Enrolment No: A2305325001

Semester: 3



GUIDE CERTIFICATE

I hereby certify that the Internship-I report by Samay Kumar, student of Amity School of Engineering & Technology, Amity University Uttar Pradesh, Noida, Enrolment No: A2305325001 with title “Centralized vs Peer-to-Peer Open-Source VPN Architectures in Linux Environments” which is submitted to Amity School of Engineering & Technology, Amity University Uttar Pradesh, Noida, in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology (Information Technology), is an original contribution and a faithful record of the work carried out by him under my guidance and supervision and to the best of my knowledge, this work has not been submitted, in part or in full, for the award of any degree or diploma to this University or elsewhere.

Place: Noida

Date:

(Signature)

Dr. Veenu Singh

Assistant Professor-II

Department of Information Technology, ASET

AUUP, Noida

ACKNOWLEDGEMENT

It is my privilege to express my deep sense of gratitude to those entire faculty Members who helped me in the completion of the Internship report, especially my guide Dr. Veenu Singh who was always there at hour of need.

My special thanks to all other faculty members, Batch mates & Seniors of Amity School of Engineering & Technology, Amity University, Uttar Pradesh for helping me in the completion of Internship work and its report submission.

Samay Kumar

Enrolment No: A2305325001

TABLE OF CONTENTS

S. No.	Topic	Page No.
1	Declaration	2
2	Guide Certificate	3
3	Acknowledgement	4
4	Abstract	6
5	Introduction	7
6	Literature Review	8
7	Methodology	10
8	Implementation and Comparative Evaluation	12
9	Result and Discussions	14
10	Future Scope	16
11	Conclusion	17
12	References	18

LIST OF TABLES

Table No.	Title of the Table	Page No.
1	VPNLens Technology Stack	12
2	Benchmark Metrics Explanations	13
3	Benchmark Metrics Collected	13

LIST OF FIGURES

Figure No.	Caption of the Figure	Page No.
1	Infrastructure Architecture	10
2	Benchmark Results	14

ABSTRACT

Virtual Private Networks (VPNs) are widely used to secure communication across distributed systems, and Linux remains one of the primary platforms for deploying such infrastructure. This report presents a comparative study of two open-source VPN deployment models, namely the centralized hub-and-spoke architecture represented by WireGuard and the peer-to-peer mesh architecture coordinated through Headscale, implemented on Ubuntu Linux virtual machines. Both architectures were deployed, configured, and validated using standard Linux networking utilities. To enable a consistent and repeatable evaluation, a benchmarking framework named VPNLens was developed. The framework automates VPN switching, executes benchmark tests, collects network and system performance metrics including latency, throughput, packet loss, CPU utilization, memory utilization, connection time, and recovery behaviour, and presents the collected data through an interactive dashboard. The study examines the practical characteristics of both VPN architectures with respect to deployment, routing behaviour, connectivity, and operational management. The observations indicate that each architecture offers distinct advantages depending on the deployment scenario, and that selecting an appropriate VPN architecture should be based on application requirements rather than a single performance metric. The framework developed during this work also provides a foundation for extending the comparative evaluation to additional open-source VPN technologies in future studies.

INTRODUCTION

Virtual Private Networks (VPNs) enable secure communication over public networks by establishing encrypted tunnels between communicating systems. As organisations increasingly adopt cloud infrastructure, remote access, and distributed services, VPNs have become a fundamental component of modern network security. Linux, owing to its mature networking stack, stability, and extensive open-source ecosystem, remains one of the most widely used platforms for deploying VPN infrastructure.

Two broad architectural models are commonly used for VPN deployment: the centralized, or hub-and-spoke, model, where a dedicated server routes traffic between connected peers, and the peer-to-peer, or mesh, model, where a coordination service handles authentication and peer discovery while traffic flows directly between endpoints. These two models, along with their respective trade-offs, are discussed in detail in the Literature Review.

To represent these deployment models, this study employs WireGuard, a kernel-level VPN protocol commonly deployed using a centralized architecture [1], [3], and Headscale, a self-hosted coordination server that enables peer-to-peer WireGuard tunnels between authenticated nodes [2], [4]. Since both solutions utilise the same underlying WireGuard protocol for encrypted communication, they provide an appropriate basis for comparing deployment architectures while minimising differences arising from the cryptographic protocol itself.

Although comprehensive documentation exists for deploying both WireGuard and Headscale individually, comparatively few practical studies evaluate these architectures under a common Linux environment using a consistent testing methodology. This internship addresses that need by deploying both VPN architectures on Ubuntu Linux virtual machines and evaluating them under identical experimental conditions. To support this evaluation, a benchmarking framework named VPNLens was developed to automate the testing workflow; its design and functionality are described in the Methodology section. The primary objective of VPNLens is to provide a common evaluation platform; it serves as a supporting framework for the comparative study rather than the focus of the project.

LITERATURE REVIEW

Early Virtual Private Network (VPN) technologies, including PPTP, L2TP/IPsec, and OpenVPN, established secure communication over public networks but were often associated with complex configuration procedures and comparatively higher computational overhead. As cloud computing, remote access, and distributed applications became more common, the focus shifted towards VPN solutions that were simpler to deploy, easier to manage, and capable of providing improved performance. Linux, supported by a mature networking stack and an extensive collection of open-source networking utilities such as `iproute2`, has consequently become one of the most widely adopted platforms for deploying modern VPN infrastructure [6].

The centralized, or hub-and-spoke, deployment model remains one of the most commonly used VPN architectures. In this model, all participating clients establish secure tunnels with a central server responsible for routing traffic between connected peers. This approach simplifies deployment, authentication, and access control, but also introduces a dependency on the central server, whose availability and processing capacity directly influence the performance and reliability of the entire network. In contrast, the peer-to-peer, or mesh, architecture enables participating nodes to establish direct encrypted tunnels after authentication and peer discovery have been completed by a coordination service. This approach reduces indirect routing and improves communication efficiency, although it introduces additional complexity in node coordination and NAT traversal.

WireGuard, introduced by Jason A. Donenfeld, was developed as a lightweight alternative to traditional VPN protocols by combining a compact codebase with a fixed set of modern cryptographic primitives, including Curve25519 for key exchange and ChaCha20-Poly1305 for authenticated encryption [1]. Its handshake mechanism is based on the Noise Protocol Framework and was designed to minimise complexity while maintaining strong security. Owing to its efficient implementation, WireGuard has been integrated directly into the Linux kernel, allowing encrypted packet processing to occur within the kernel networking stack [3]. Although WireGuard is frequently deployed using a centralized topology, the protocol itself is independent of any specific deployment architecture and can also be configured for site-to-site or mesh networks.

Headscale is an open-source, self-hosted implementation of the Tailscale coordination server that enables organisations to deploy private control-plane infrastructure without relying on external services [2], [4]. Unlike traditional VPN servers, Headscale does not forward user traffic. Instead, it authenticates participating nodes, distributes cryptographic keys, and coordinates peer discovery so that devices can establish direct WireGuard tunnels with one another. Consequently, the control plane remains centralized, while the encrypted data plane operates through direct peer-to-peer

communication. This separation between coordination and data transmission represents one of the principal architectural differences examined in this study.

Previous studies have commonly evaluated VPN performance using standard Linux networking tools such as ping for latency measurement, iperf3 for throughput testing, and system monitoring utilities for observing CPU and memory utilisation [5]. While these tools provide reliable performance measurements individually, benchmarking multiple VPN deployments often requires repeated manual configuration, execution, and result collection. Such an approach can be time-consuming and may introduce inconsistencies between successive evaluations. To address this practical limitation, this internship developed the VPNLens framework, described in detail in the following section.

METHODOLOGY

The methodology adopted during this internship consisted of five stages: literature study, environment preparation, VPN deployment, benchmarking framework development, and performance evaluation. Each stage contributed to the systematic comparison of centralized and peer-to-peer VPN architectures under a common Linux environment.

Literature Study

The initial stage involved studying the architectural principles of centralized and peer-to-peer VPN deployments, together with the operating mechanisms of WireGuard and Headscale. Official documentation, protocol specifications, and relevant technical references were reviewed to understand their deployment models, routing behaviour, authentication mechanisms, and implementation within Linux environments [1]–[4]. This study established the technical foundation for the practical implementation carried out during the internship.

Environment Preparation

The experimental environment was prepared using Ubuntu 22.04 LTS virtual machines deployed on cloud infrastructure. Each virtual machine was configured with SSH access, appropriate firewall rules, IP forwarding, and the networking utilities required for VPN deployment and benchmarking. This ensured that both VPN architectures were evaluated under a consistent software environment.

VPN Deployment

WireGuard was deployed using a centralized hub-and-spoke architecture. Cryptographic key pairs were generated for each participating node, configuration files were created to define peer relationships, allowed IP ranges, and communication endpoints, and IP forwarding was enabled on the central server to facilitate packet routing.

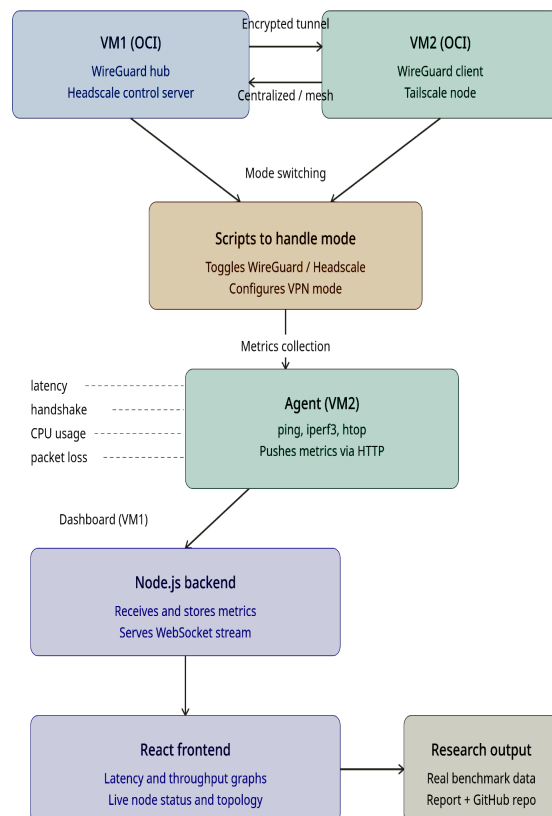


Fig 1: Infrastructure Architecture

Headscale was deployed as a self-hosted coordination server supporting a peer-to-peer architecture. Client nodes were registered using the Tailscale client, after which authenticated peers established direct WireGuard tunnels with one another. Before benchmarking, both deployments were validated using standard Linux networking utilities such as ping, wg show, and ip route to verify tunnel establishment, routing configuration, and end-to-end connectivity.

Benchmark Framework Development

To ensure a consistent evaluation methodology, a benchmarking framework named VPNLens was developed during the internship. The framework automates VPN switching, executes a predefined sequence of benchmark tests, collects network and system performance metrics, stores the generated results, and presents them through an interactive dashboard. Automating the benchmarking process reduced manual intervention, improved repeatability, and ensured that both VPN architectures were evaluated under comparable conditions.

Performance Evaluation

Following successful deployment and validation, benchmark tests were executed using the VPNLens framework. The evaluation focused on commonly used networking and system performance metrics, including latency, throughput, packet loss, CPU utilisation, memory utilisation, connection time, and recovery behaviour. The collected observations were then used to compare the operational characteristics of the centralized and peer-to-peer VPN architectures. The implementation details and the resulting observations are presented in the following section.

IMPLEMENTATION AND COMPARATIVE EVALUATION

Implementation Overview

The experimental environment was implemented using Ubuntu 22.04 LTS virtual machines configured with identical operating system settings to ensure a consistent basis for comparison. Both VPN deployments, configured as outlined in the Methodology, were verified using standard Linux networking utilities before benchmark execution to ensure that all communication occurred through the intended VPN tunnel.

The comparative evaluation was supported by the VPNLens framework described in the Methodology, which automated tunnel validation, benchmark execution, and result collection for both deployments.

Component	Technology Used
Operating System	Ubuntu 22.04 LTS
Centralized VPN Protocol	WireGuard
Peer-to-Peer Control Plane	Headscale
Containerization	Docker
Backend Orchestration	Node.js + Express
Dashboard / Visualization	React
Data Persistence	SQLite
Benchmark Utilities	ping, iperf3, Linux monitoring tools

Table 1: VPNLens Technology Stack

Performance Metrics

The benchmarking framework collected a set of networking and system performance metrics to enable a consistent comparison between the two VPN architectures. These metrics were selected because they provide practical insight into network behaviour, resource utilisation, and connection characteristics during VPN operation.

Latency and packet loss were measured using ping, while throughput measurements were obtained using iperf3. CPU and memory utilisation were monitored during benchmark execution using standard Linux monitoring tools. Connection establishment and recovery behaviour were recorded during tunnel initialization and interface restoration. Collectively, these metrics provided a common basis for evaluating the operational characteristics of the two deployment architectures.

Metric	Description
Latency	Round-trip time for packets to travel between communicating nodes.
Throughput	Sustained rate of data transfer across the VPN tunnel (upload and download).
Packet Loss	Percentage of packets lost during communication.
CPU Utilization	Processor resources consumed while the tunnel is under active load.
Memory Utilization	Memory footprint during benchmark execution.
Connection Time	Time required to bring up and verify a VPN tunnel.
Recovery Time	Time required to resume routing after a deliberate interface disruption.

Table 2: Benchmark Metrics Explanations

Data Collection, Analysis and Comparative Evaluation

Following successful deployment and verification of both VPN architectures, benchmark experiments were executed using the VPNLens framework. Each benchmark followed an identical workflow consisting of VPN selection, tunnel verification, performance testing, metric collection, result storage, and dashboard visualization. Executing the same sequence for both deployments ensured that the evaluation methodology remained consistent throughout the study.

The collected benchmark data included latency, throughput, packet loss, CPU utilisation, memory utilisation, connection time, and recovery behaviour. These observations were analysed to identify practical differences between the centralized and peer-to-peer deployment models while maintaining the same Linux environment and benchmarking procedure. Rather than focusing on a single performance metric, the evaluation considered the combined behaviour of multiple networking and system characteristics.

The benchmarking framework successfully automated the complete evaluation process and demonstrated that both VPN architectures could be deployed, tested, and monitored using a unified workflow. The collected observations provide a practical basis for comparing the operational characteristics of centralized and peer-to-peer VPN deployments under identical testing conditions.

RESULTS AND DISCUSSION

The benchmark experiments were conducted under identical deployment conditions using the VPNLens framework. Performance observations were obtained by executing repeated benchmark runs for both VPN deployments and analysing the collected networking and system metrics.

The results indicate that WireGuard consistently achieved higher upload and download throughput than the Headscale-managed deployment throughout the benchmark runs. Average upload throughput was approximately 110 Mbps for WireGuard compared with 82 Mbps for Headscale, while average download throughput was approximately 120 Mbps and 88 Mbps, respectively. These observations suggest that the centralized deployment achieved higher data transfer rates within the experimental environment.

Average latency remained comparable for both deployment models. WireGuard recorded an average latency of 1.77 ms, while the Headscale-managed deployment recorded 2.04 ms. Both architectures maintained zero packet loss during the benchmark runs, indicating stable connectivity throughout the evaluation.

System resource utilisation also remained comparable. WireGuard recorded a slightly lower average CPU utilisation (20.5%) than the Headscale-managed deployment (23.9%), while average memory consumption remained close to 300 MB for both deployments. The observed differences were relatively small and are unlikely to have a substantial impact under the tested workload.

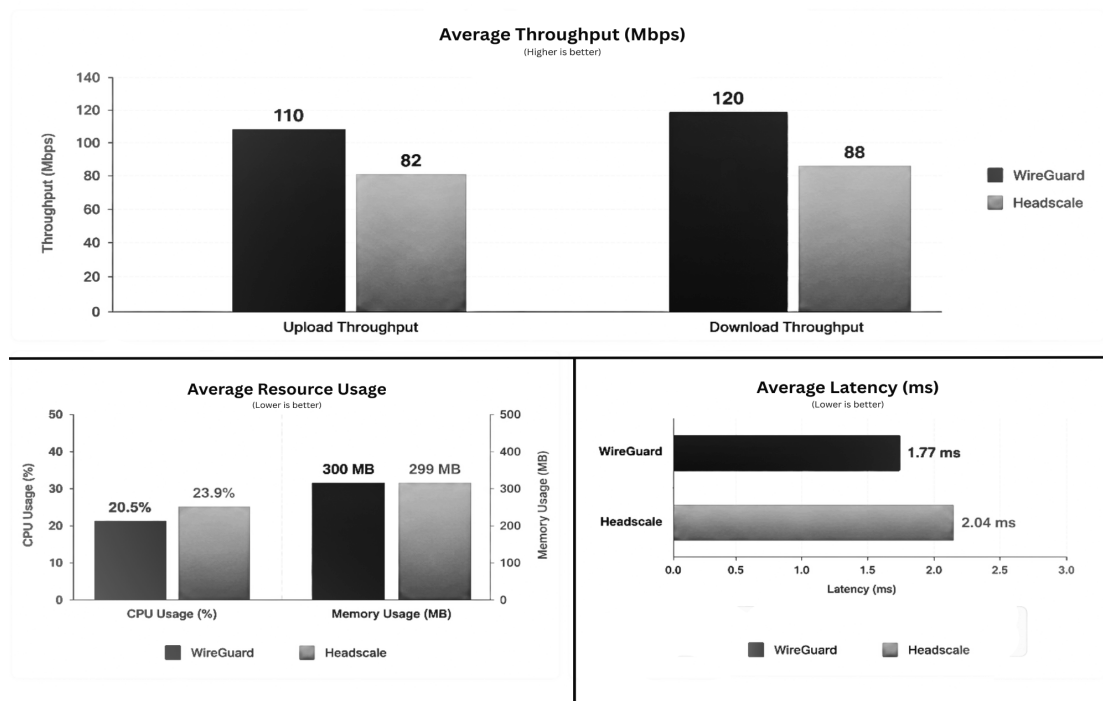


Fig 2: Benchmark Results

Connection establishment times were also similar, with WireGuard averaging 1.87 seconds and the Headscale-managed deployment averaging 2.32 seconds. Recovery time was recorded for WireGuard, reflecting kernel-level tunnel restoration, but was not meaningful for Headscale, since recovery there reflects the tailscaled service reconnecting rather than the tunnel itself. As the two do not measure the same layer, recovery was excluded from the comparison."

Overall, the benchmark results indicate that the centralized WireGuard deployment provided higher throughput while maintaining latency and resource utilisation comparable to the Headscale-managed deployment within the experimental environment. These findings represent the behaviour of the evaluated deployments under the configured experimental conditions rather than universal performance characteristics of the underlying VPN architectures; the factors influencing this are discussed further in the Limitations section.

Metric	WireGuard	Headscale
Avg Latency	1.77 ms	2.04 ms
Upload Throughput	110 Mbps	82 Mbps
Download Throughput	120 Mbps	88 Mbps
CPU Usage	20.5%	23.9%
Memory Usage	300 MB	299 MB
Packet Loss	0%	0%
Connection Time	1.87 s	2.32 s

Table 3: Benchmark Metrics Collected

Limitations

The experimental evaluation was conducted within a controlled cloud-based Linux environment using a limited number of virtual machines. Network performance is influenced by factors such as cloud infrastructure, geographic server location, network congestion, and system load. Therefore, the observations should be interpreted within the context of the experimental environment rather than as universal performance characteristics.

The study was limited to two open-source VPN solutions and did not consider other VPN technologies, large-scale deployments, or multi-client environments. In addition, the benchmarking process relied on synthetic network traffic generated using standard Recovery timetesting utilities, which may not fully represent real-world application workloads.

FUTURE SCOPE

The comparative study presented in this report can be extended by evaluating additional open-source VPN technologies that employ different deployment architectures and networking approaches. Solutions such as OpenVPN, NetBird, ZeroTier, and other self-hosted VPN platforms can be included to provide a broader understanding of secure networking within Linux environments.

Future work may also involve conducting comparative evaluations across multiple cloud providers, geographic regions, and network conditions to better understand how deployment environments influence VPN performance and operational behaviour. Increasing the number of benchmark executions and analysing larger datasets would improve the reliability of the observations and support more comprehensive comparisons.

The scope of evaluation can be expanded by incorporating additional networking metrics such as jitter, handshake latency, NAT traversal success rate, connection stability, scalability, and performance under increasing numbers of connected peers. These parameters would provide a more comprehensive assessment of the operational characteristics of different VPN architectures.

Another possible direction is the automation of infrastructure provisioning using Infrastructure as Code (IaC) tools such as Terraform and configuration management platforms such as Ansible. Automating the deployment of Linux environments and VPN configurations would improve reproducibility, simplify large-scale experimental studies, and support more consistent comparative evaluations.

Overall, this study provides a practical foundation for future comparative evaluation of centralized and peer-to-peer open-source VPN architectures in Linux environments. Expanding the range of VPN technologies, deployment scenarios, and experimental datasets would enable a more comprehensive understanding of their operational characteristics, limitations, and suitability for different networking requirements.

CONCLUSION

This internship presented a comparative study of centralized and peer-to-peer open-source VPN architectures in Linux environments using WireGuard and Headscale as representative implementations. Both VPN architectures were successfully deployed on Ubuntu Linux virtual machines and evaluated under a common experimental environment. To support a consistent evaluation methodology, a benchmarking framework named VPNLens was developed to automate VPN switching, benchmark execution, metric collection, and result visualization.

The comparative evaluation demonstrated that the two deployment models exhibit different operational characteristics that arise from their underlying architectural design, as discussed in the Literature Review. These differences highlight that the selection of a VPN architecture should be based on deployment requirements, operational considerations, and network design objectives rather than a single performance metric.

Overall, this work provides a practical comparison of two widely used open-source VPN deployment models within a Linux environment while establishing a repeatable methodology for their evaluation, with further extensions outlined in the Future Scope section.

REFERENCES

- [1] J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” in Proc. NDSS Symp., 2017. [Online]. Available: <https://www.wireguard.com/papers/wireguard.pdf>
- [2] Tailscale Inc., “How Tailscale Works,” Tailscale Engineering Blog, 2020. [Online]. Available: <https://tailscale.com/blog/how-tailscale-works>
- [3] Linux Kernel Organization, “WireGuard – Fast, Modern, Secure VPN Tunnel,” The Linux Kernel Documentation. [Online]. Available: <https://www.kernel.org/doc/html/latest/networking/wireguard.html>
- [4] juanfont, “Headscale: An open source Tailscale control server,” GitHub, 2021. [Online]. Available: <https://github.com/juanfont/headscale>
- [5] J. Rosen, “A Comparison of WireGuard, IKEv2/IPsec, and OpenVPN,” Mullvad VPN Blog, 2020. [Online]. Available: <https://mullvad.net/en/blog/>
- [6] Linux Man-Pages Project, “ip-rule – routing policy database management,” Linux man-pages. [Online]. Available: <https://man7.org/linux/man-pages/man8/ip-rule.8.html>
- [7] Canonical Ltd., “Ubuntu Server Guide,” Ubuntu Documentation Team. [Online]. Available: <https://ubuntu.com/server/docs>
- [8] Docker Inc., “Docker Documentation.” [Online]. Available: <https://docs.docker.com/>
- [9] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: <https://nodejs.org/en/docs>
- [10] Meta Platforms Inc., “React Documentation.” [Online]. Available: <https://react.dev/>
- [11] ESnet, “iperf3 Documentation.” [Online]. Available: <https://iperf.fr/>
- [12] SQLite Consortium, “SQLite Documentation.” [Online]. Available: <https://www.sqlite.org/docs.html>

APPENDIX A

Project Resources

The implementation developed as part of this internship has been maintained separately from this report to support future development, reproducibility, and further comparative studies. The project resources associated with this work are listed below.

Resource	Details
Project Name	VPNLens
Project Description	Automated benchmarking framework for the comparative evaluation of centralized and peer-to-peer open-source VPN architectures in Linux environments.
Source Code Repository	https://github.com/samay15jan/VPNLens
Project Website	https://vpn.samay15jan.com
Project Documentation	https://docs.vpn.samay15jan.com
Benchmark Dataset (JSON)	https://github.com/samay15jan/VPNLens/tree/main/documentation/docs/benchmark-data

Note:-

The implementation provided in the repository represents the practical component developed during this internship. The benchmark results and observations discussed in this report correspond to the experimental environment described in the Methodology and Implementation sections. Future updates to the repository may include additional VPN technologies, Infrastructure as Code (IaC) support, expanded benchmarking capabilities, and further experimental datasets.